

Wednesday — Manual Exploitation

NFS Misconfiguration

Understanding a real-world protocol vulnerability by exploiting it manually, without automation frameworks such as Metasploit.

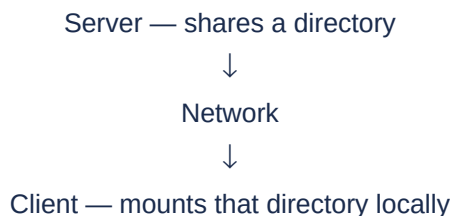
Goal

The objective was to manually exploit a vulnerability without using Metasploit. The focus was on understanding *why* the attack works, rather than relying on automation.

Unlike Tuesday, where Metasploit automated the exploitation process, every step today was performed manually so the underlying networking and protocol behavior became clear.

What Is NFS?

NFS (Network File System) is a protocol that allows one Linux/Unix machine to share directories with another machine over the network. Instead of copying files, the client mounts a remote directory so it behaves like a local folder.



After mounting, applications such as `ls`, `cat`, and `vim` do not know the files are remote — they simply ask the operating system to open files, and the Linux kernel transparently communicates with the NFS server.

Choosing the Vulnerability

NFS was selected as today's target because:

- It was discovered during Monday's enumeration.
- It can be exploited manually.
- It demonstrates a configuration mistake rather than a software vulnerability.
- It teaches how remote file systems actually work.

Understanding the Vulnerability

The vulnerability was **not** inside NFS itself — it was a **server misconfiguration**. The NFS server exported:

```
/ *
```

Breaking this down:

/ — the root of the entire Linux filesystem. Everything exists underneath it:

```
/
███ etc
███ home
███ root
███ usr
███ var
███ ...
```

***** — means "allow any client to mount this export."

Therefore the server was effectively saying: *"I am willing to share my entire filesystem with anyone that can reach me."* No software bug was exploited — the server voluntarily granted access because its configuration explicitly allowed it.

Manual Enumeration

Command

```
showmount -e 10.174.253.161
```

Purpose: ask the NFS server, "What directories are you exporting?"

Output

```
Export list for 10.174.253.161:  
/ *
```

Conclusion: the target was exporting its entire filesystem to every client.

Mounting the Remote Filesystem

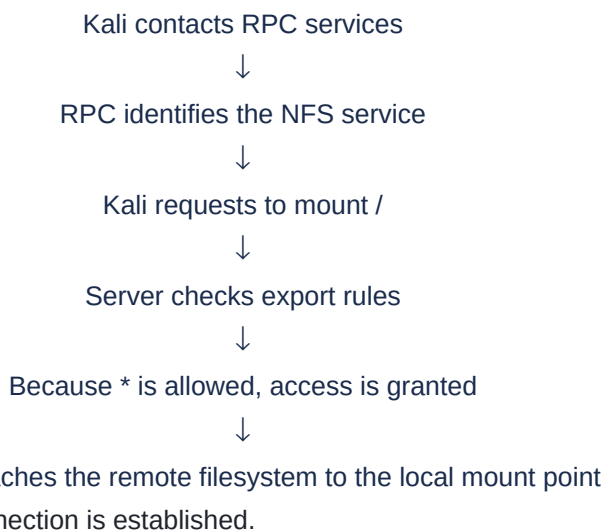
A local mount point already existed at ~/nfs.

```
sudo mount -t nfs 10.174.253.161:/ ~/nfs
```

Meaning of each part:

- sudo — administrative privileges.
- mount — attach a filesystem.
- -t nfs — filesystem type.
- 10.174.253.161:/ — mount the remote root directory.
- ~/nfs — where it appears locally.

Internally, this sequence occurs:



rpc-statd

During the first mount, Kali automatically enabled `rpc-statd`. This was initially confusing, but an important conclusion followed: **rpc-statd is not related to exploitation.**

Its purpose is:

- NFS file locking.
- Crash recovery.
- Lock cleanup after client/server failures.

It does **not**:

- Keep NFS mounted after reboot.
- Automatically reconnect mounts.

Persistent mounts are instead configured through `/etc/fstab`.

Verifying the Mount

```
ls ~/nfs
```

```
bin
boot
etc
home
root
usr
var
...
```

This proved that ~/nfs was no longer an ordinary folder — it represented the victim filesystem mounted into ~/nfs.

Reading Remote Files

```
cat ~/nfs/etc/passwd
```

Important realization: the file was **not** Kali's own /etc/passwd — it was the victim's /etc/passwd.

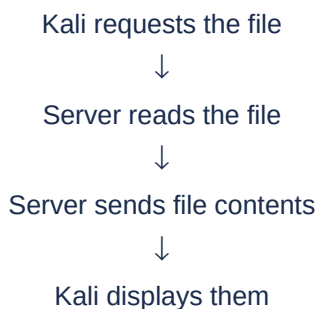
This file revealed:

- Usernames.
- Home directories.
- Shells.
- Installed services.

Password hashes were absent because modern Linux stores them inside /etc/shadow.

How NFS Actually Reads Files

Key question: "If Kali never stored the files, how could it display them?" Answer: Kali requests data only when needed — e.g. `cat ~/nfs/etc/passwd`.



The file always physically remains on the server.

RAM and Filesystem Cache

The file **does** exist temporarily on Kali — not permanently. RAM and the Linux page cache / filesystem cache may temporarily store recently accessed data. This is similar to a web browser.

Browser	NFS
Requests webpage	Requests file
Receives HTML	Receives data
Stores temporarily	Caches temporarily
Displays page	Displays file

Difference: a browser understands webpages, while NFS makes remote files appear as ordinary local files.

Mount Persistence

Question: "What happens if I never unmount?" **Answer:** as long as Kali remains running, both read and write access continue.

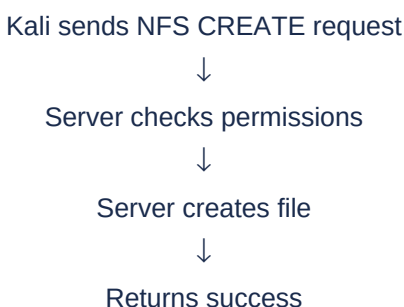
If Kali reboots, the mount disappears — mount information exists only while the operating system is running. After reboot, `~/nfs` still exists as a normal directory, but the remote filesystem does not.

To make mounts survive a reboot, `/etc/fstab` must contain an NFS entry.

Testing Write Access

```
touch ~/nfs/tmp/testifile
```

Purpose: attempt to create a file on the remote server. Internally:



The file was created on the **victim**, not on Kali.

Understanding Ownership

```
ls -l ~/nfs/tmp/testifile
```

```
himal himal
```

Conclusion: NFS stores numeric UIDs/GIDs. Kali translated UID 1000 into `himal`. If another Linux machine had UID 1000 assigned to "alice", that exact same file would appear owned by "alice".

Displayed username depends on the client — the stored UID remains the same.

Cleaning Up

```
rm ~/nfs/tmp/testifile
```

```
sudo umount ~/nfs
```

This deletes the remote file and disconnects the mounted filesystem. Unmounting removes only the connection — it does not delete server files.

Comparing With Tuesday

	Tuesday	Wednesday
Target	vsFTPD 2.3.4	NFS
Vulnerability type	Software vulnerability	Configuration vulnerability
Tooling	Metasploit	Manual
Process	Triggered exploit, opened shell, managed payload, automated everything	Enumerate exports, interpret configuration, mount filesystem, read files, verify write access, clean up

The learning value was significantly higher on Wednesday because every protocol step was understood, rather than abstracted away by automation.

Defender Perspective

Proper NFS configuration should:

- Export only necessary directories.
- Never export /.
- Restrict trusted client IPs.
- Enable root_squash.
- Protect NFS with firewall rules.

Instead of:

```
/ *
```

Use:

```
/shared 192.168.1.50
```


Understanding root_squash

Initially there was confusion about why root_squash exists if administrators can already restrict client access. The final understanding is that there are two different security problems:

1. Who is allowed to connect?	2. What privileges should a connected client receive?
Solved by: • Export rules • Firewall • Allowed IP addresses	Solved by: root_squash

Even if a trusted client becomes compromised and the attacker gains root on that client:

Without root_squash	With root_squash
Remote UID 0 → Server treats it as local root.	Remote UID 0 → Mapped to "nobody".

Therefore, the NFS server never automatically trusts remote root. A firewall cannot solve this problem because firewalls only inspect network information such as IP addresses, ports, and protocols — they do **not** understand NFS user IDs. The NFS server itself must enforce this policy.

Final Conclusions

- NFS vulnerabilities are often configuration mistakes rather than software bugs.
- The server voluntarily exposed its filesystem because of insecure exports.
- Mounting creates a live connection to remote files; it does not copy them permanently.
- Files are retrieved on demand and temporarily cached by the client.
- Rebooting removes the mount unless configured through /etc/fstab.
- UID ownership shown by the client is only a local interpretation of numeric IDs.
- root_squash protects the server from trusting remote root users, even if the connecting client is compromised.
- Manual exploitation provided a much deeper understanding of the NFS protocol than an automated exploitation framework could.